

# Structures itératives

Les boucles sont des structures itératives qui permettent de répéter un bloc de code tant qu'une condition est vraie. Elles sont essentielles pour automatiser des tâches répétitives et manipuler des listes de données.

On entend par **itération** le fait de répéter une action ou un ensemble d'actions. Les structures itératives permettent de parcourir des éléments, de les modifier ou de les traiter de manière répétée.

## Boucles **for**

La boucle **for** est utilisée pour répéter une action un nombre fixe de fois. En pseudo-code, elle se présente ainsi :

```
POUR i DE 0 À 9 FAIRE
    // actions à répéter
FIN POUR
```

1. On écrit **for** pour indiquer le début de la boucle, ensuite on place une paire de parenthèses **()** qui contiendra trois parties séparées par des points-virgules **;** et enfin on ouvre une accolade **{** pour débiter le bloc de code à répéter.
2. On écrit **let i = 0** pour initialiser la variable **i** à 0, c'est notre point de départ.
3. Ensuite, **i < 10** pour définir la condition d'arrêt de la boucle, c'est-à-dire que la boucle continuera tant que **i** est inférieur à 10 dans cet exemple.
4. Finalement, on utilise **i++** pour incrémenter (augmenter) **i** de 1 à chaque passage dans la boucle.

**Attention** On termine toujours une instruction par un point-virgule **;** en JavaScript. Par exemple, pour afficher un message dix fois :

```
for (let i = 0; i < 10; i++) {
    console.log("Bonjour !");
}
```

## Exemple

À votre avis, que fais le code suivant ?

```
for (let i = 10; i >= 0; i--) {
    console.log(i);
}
```

En effet, ce code affiche les nombres de 10 à 0 en décrémentant (diminuant) **i** à chaque itération.

## Boucles **while**

La boucle `while` est utilisée lorsque le nombre d'itérations n'est pas connu à l'avance et dépend d'une condition. En pseudo-code, elle se présente ainsi :

```
TANT QUE condition FAIRE
    // actions à répéter
FIN TANT QUE
```

1. On écrit `while` pour indiquer le début de la boucle, ensuite on place une paire de parenthèses `()` qui contiendra la condition d'arrêt et enfin on ouvre une accolade `{` pour débiter le bloc de code à répéter.
2. La boucle continue tant que la condition est vraie. Par exemple, pour afficher un message tant qu'une variable est inférieure à 5 :

```
let i = 0;
while (i < 5) {
    console.log("Bonjour !");
    i++; // N'oubliez pas d'incrémenter i pour éviter une boucle infinie
}
```

## Exemple en jeu

Pour poser une question à l'utilisateur jusqu'à ce qu'il réponde correctement, on peut utiliser une boucle `while` car on ne sait pas combien de fois l'utilisateur va se tromper. Si la réponse est vide, on continue à poser la question :

```
let reponse = ""; // Initialisation de la variable

while (reponse == "") {
    reponse = prompt("Quelle est la capitale de la France ?");
}
//...reste du code
```

## Portée des variables et blocs d'exécution

Attention à la portée des variables déclarées avec `let` ou `const`. Une variable déclarée à l'intérieur d'une boucle n'est pas accessible en dehors de cette boucle. Cela est pratique pour éviter les conflits de noms, mais il faut en être conscient.

```
for (let i = 0; i < 5; i++) {
    let message = "Itération " + i;
    console.log(i); // i est accessible ici
}

// console.log(message); // Erreur : message n'est pas accessible ici
```

Pour remédier à cela, on peut déclarer la variable en dehors de la boucle si on a besoin de l'utiliser après la boucle.

```
let message; // Déclaration en dehors de la boucle
for (let i = 0; i < 5; i++) {
  message = "Itération " + i;
  console.log(i); // i est accessible ici
}

console.log(message); // message est accessible ici
```